

# Harris Hawks Optimization

## Algorithm Overview and Adaptation to Combinatorial Optimization Problems

Hamdan Kaouthar

June 29, 2026

# Outline

- 1 Background
- 2 Mathematical Modeling of HHO
- 3 Big Picture of the Algorithm
- 4 Target Problem: Combinatorial Optimization
- 5 Adapted HHO for Combinatorial Optimisation
- 6 Conclusion

# Nature-Inspired Background: Harris Hawks Hunting

Harris's hawk is considered one of the most intelligent birds of prey. This raptor from the American desert is known for a unique cooperative hunting strategy within stable groups, unlike most raptors that hunt alone.

Its strategy relies on coordinated phases:

- Detecting and surrounding the prey, often a rabbit.
- Attacking simultaneously from several directions: a “surprise pounce”.
- Taking turns if the leading hawk fails.
- Pursuing the prey until it becomes exhausted.



## Core idea

Harris Hawks Optimization (HHO) is a population-based, gradient-free optimization algorithm inspired by cooperative hunting.

- **Hawks** represent candidate solutions in the search space.
- **Rabbit** represents the current best solution, or optimum.
- The algorithm alternates between wide search and focused attack.
- This balance is controlled by the prey's escaping energy  $E$ .

# Exploration Phase: Searching for Prey

Hawks randomly perch in two different ways, controlled by probability  $q$ .

$q \geq 0.5$ : Random tall trees

$$X(t+1) = X_{rand} - r_1 |X_{rand} - 2r_2 X(t)|$$

$q < 0.5$ : Near group and rabbit

$$X(t+1) = (X_{rabbit} - X_m) - r_3 (LB + r_4 (UB - LB))$$

- $X_m$  is the average position of all hawks: the group's center.
- $r_1, r_2, r_3, r_4$  are random numbers in  $(0, 1)$ .

## Escaping energy of the prey

$$E = 2E_0 \left(1 - \frac{t}{T}\right)$$

### Meaning

- $E_0 \in (-1, 1)$  is randomly updated each iteration.
- $t$  is the current iteration.
- $T$  is the maximum number of iterations.

### Decision rule

- $|E| \geq 1$ : exploration, hawks still search.
- $|E| < 1$ : exploitation, hawks attack.

# Exploitation Phase: The Surprise Pounce

Four attacking strategies are selected using the escaping energy  $|E|$  and the prey's escape chance  $r$ .

| Condition                  | Strategy                   | Key equation                                   |
|----------------------------|----------------------------|------------------------------------------------|
| $r \geq 0.5,  E  \geq 0.5$ | Soft besiege               | $X(t+1) = \Delta X(t) - E JX_{rabbit} - X(t) $ |
| $r \geq 0.5,  E  < 0.5$    | Hard besiege               | $X(t+1) = X_{rabbit} - E \Delta X(t) $         |
| $r < 0.5,  E  \geq 0.5$    | Soft besiege + rapid dives | Best of $Y$ or $Z$ using Lévy Flight           |
| $r < 0.5,  E  < 0.5$       | Hard besiege + rapid dives | Best of $Y$ or $Z$ using $X_m$                 |

Lévy Flight mimics the rabbit's zigzag escape and the hawks' rapid, irregular dives.

# HHO Algorithm Flow

- ➊ Initialize  $N$  hawks randomly.
- ➋ Repeat until the stopping condition is met:
  - Evaluate fitness and set the **Rabbit** as the best hawk.
  - Update the escaping energy  $E$ .
  - If  $|E| \geq 1$ , use **exploration**.
  - If  $|E| < 1$ , use one exploitation strategy:
    - Soft besiege:  $r \geq 0.5$ ,  $|E| \geq 0.5$ .
    - Hard besiege:  $r \geq 0.5$ ,  $|E| < 0.5$ .
    - Soft + Lévy dives:  $r < 0.5$ ,  $|E| \geq 0.5$ .
    - Hard + Lévy dives:  $r < 0.5$ ,  $|E| < 0.5$ .
- ➌ Return the best solution: the Rabbit location.

# Problem to Solve: TSP and VRP

## Combinatorial optimization problem

The goal is to find the best arrangement among a very large number of possible discrete solutions.

### Travelling Salesman Problem (TSP)

- Visit each city exactly once.
- Return to the starting city.
- Minimize the total travel distance.

### Vehicle Routing Problem (VRP)

- Serve a set of customers from a depot.
- Respect vehicle capacity constraints.
- Minimize the total route cost.

# Why Standard HHO Cannot Be Applied Directly

## HHO operates in continuous space

Hawks move by arithmetic equations:

$$X(t+1) = X_{\text{rabbit}} - E |J \cdot X_{\text{rabbit}} - X(t)|$$

These require subtraction, multiplication, and absolute value on **real-valued** vectors.

e.g. [ -0.8, 0.3, -0.1, 0.9, 0.5 ]

## TSP / VRP require discrete permutations

A valid solution must:

- Visit each city **exactly once**
- Contain only integers, no real values
- Preserve order to define route cost

e.g. [ 3, 1, 5, 2, 4 ]

## Core challenge

How can HHO's continuous movement explore a discrete permutation space?

# The Proposed Solution: A Transformation Layer

Each solution is represented simultaneously in **two spaces**.

## Discrete space

Permutation of integers

[ 3, 1, 5, 2, 4 ]

- Represents a route
- Each value appears once
- Used for fitness evaluation only

forward  
→

← backward + repair

*Transformation layer*  
(Davendra & Onwubolu)

## Continuous space

Real-valued vector in  
[lb, ub]

[-1.0, -0.5, 1.0, 0.5]

- HHO equations applied here
- Exploration and exploitation
- No validity constraint

# Forward Transformation: Permutation $\rightarrow$ Continuous Vector

## Mapping formula (Davendra & Onwubolu)

$$z_i = lb + (ub - lb) \cdot \frac{x_i - 1}{n - 1}$$

where  $x_i$  is the integer at position  $i$ ,  $n$  is the dimension, and  $[lb, ub]$  are the continuous bounds (default  $-1$  to  $1$ ).

## Properties:

- City 1 always maps to  $lb$ ; city  $n$  always maps to  $ub$ .
- The mapping is **order-preserving**: a higher city index yields a higher continuous value.
- HHO can now apply subtraction and multiplication freely.

**Worked example** —  $n = 5$ ,  $lb = -1$ ,  $ub = 1$ :

| Position $i$      | 1    | 2     | 3    | 4     | 5    |
|-------------------|------|-------|------|-------|------|
| Permutation $x_i$ | 3    | 1     | 5    | 2     | 4    |
| Continuous $z_i$  | 0.00 | -1.00 | 1.00 | -0.50 | 0.50 |

# Backward Transformation and Repair

## Step 1 — Decode: real value $\rightarrow$ integer position

$$x_i = \text{round} \left[ \frac{z_i - lb}{ub - lb} \cdot (n - 1) \right] + 1$$

## Step 2 — The problem: duplicates

HHO may push values close together, so rounding produces the same integer more than once.

*Continuous:*

$[-0.6, -0.4, 0.8, -0.5, 0.5]$

*Decoded:*

$[2, 2, 5, 2, 4]$

City 2 appears three times.

## Step 3 — Repair: cheapest insertion

For each missing city, try inserting it at every duplicate position and keep the placement that gives the **lowest objective value**.

Missing:  $\{1, 3\}$

Duplicates at positions:  $i = 1, i = 3$

*Result:*

$[1, 2, 5, 3, 4] \checkmark$

# The Complete Adapted HHO Loop

- ➊ **Initialise** a population of valid random permutations.
- ➋ **Forward transform** each permutation into a continuous vector:  $[1..n] \rightarrow [lb, ub]$ .
- ➌ **Run HHO** normally in continuous space:
  - Exploration phase if  $|E| \geq 1$ .
  - Soft / hard besiege or Lévy dive if  $|E| < 1$ .
- ➍ **Backward transform** the new continuous vector back to integers.
- ➎ **Repair** the decoded solution to restore a valid permutation (cheapest-insertion).
- ➏ **Evaluate** the objective function on the permutation.
- ➐ **Update** the hawk's position if the trial solution improves its fitness.
- ➑ Repeat from step 2 until the maximum number of iterations is reached.

Steps 2 and 4–5 are the only additions to standard HHO. The algorithm core is completely unchanged.

## Main contribution

This work adapts Harris Hawks Optimization to permutation-based combinatorial problems such as TSP and VRP.

- Standard HHO is continuous, while TSP and VRP require discrete routes.
- The forward–backward transformation connects both representations.
- The repair step guarantees valid permutations after decoding.
- The HHO core remains unchanged and can be reused for other problems.

**Final idea:** continuous search + discrete repair = adapted HHO.

# Thank you!

Questions?

Hamdan Kaouthar